🔁 Ciclo for en Python

contadores · acumuladores · range() · recorrer textos · ciclos anidados

Lógica de Programación 1 · Guía del estudiante

Muchos problemas repiten **lo mismo muchas veces**: liquidar el pago de cada empleado, revisar cada letra de una contraseña, calcular la cuota de cada mes de un crédito. Un **ciclo** repite un bloque de instrucciones sin tener que copiarlo. El ciclo `for` se usa cuando **sabes cuántas vueltas** habrá o cuando **recorres una colección** (un texto, una lista).

1 · El ciclo for

nomina.py

```
for i in range(1, 6):
    print(f"Empleado {i}: pago registrado")
print("Nómina lista")
```

```
Empleado 1: pago registrado
Empleado 2: pago registrado
Empleado 3: pago registrado
Empleado 4: pago registrado
Empleado 5: pago registrado
Nómina lista
```

Parte	Significado
`for`	"para cada..."
`i`	variable que toma un valor distinto en cada vuelta
`in`	"...en..." (obligatoria)
`range(1, 6)`	la secuencia: 1, 2, 3, 4, 5
`:` + sangría	el **cuerpo**: se repite en cada vuelta
sin sangría	después del ciclo: se ejecuta una sola vez

PSeInt	Python	Ojo
`Para i <- 1 Hasta 5 Hacer`	`for i in range(1, 6):`	la parada **no se incluye**: para llegar a 5 se escribe 6
`Para i <- 0 Hasta 10 Con Paso 2 Hacer`	`for i in range(0, 11, 2):`	el paso va de tercero
`Para i <- 5 Hasta 1 Con Paso -1 Hacer`	`for i in range(5, 0, -1):`	hacia atrás: paso negativo, parada un número antes del final
`FinPara`	(desaparece)	la sangría marca dónde termina el cuerpo
`c <- c + 1`	`c += 1`	también `-=`, `*=`, `/=`

2 · La función range()

`range(inicio, parada, paso)` genera enteros: empieza en **inicio** (por defecto 0), avanza de a **paso** (por defecto 1, puede ser negativo, nunca 0) y se detiene **antes** de la **parada**.

Escribes	Genera	Vueltas
`range(4)`	`0 1 2 3`	4
`range(3, 8)`	`3 4 5 6 7`	5
`range(5, 26, 5)`	`5 10 15 20 25`	5
`range(9, 0, -3)`	`9 6 3`	3
`range(6, 2)`	(nada)	0: con paso +1 ya empieza después de la parada

> **Truco:** con paso 1, `range(a, b)` da exactamente **b – a** vueltas. Si `n` viene de `input()`, conviértelo con `int()`: `range()` no acepta números con decimales (**TypeError**).

3 · Contadores y acumuladores

	Contador	Acumulador
¿Qué guarda?	cuántas veces ocurrió algo	la suma (o el producto) de valores que cambian
¿Cómo cambia?	una cantidad fija: `c += 1`	un valor que cambia: `total += precio`
¿Con qué empieza?	`0`	`0` si suma (0 + x = x) · `1` si multiplica (1 × x = x)
¿Dónde se crea?	**Antes** del ciclo. Si se crea dentro, se reinicia en cada vuelta.	

Contador: días activos de la semana (8000 pasos o más)

pasos.py

```
dias_activos = 0  # contador: empieza en 0
for dia in range(1, 8):  # 7 vueltas
    pasos = int(input(f"Pasos del día {dia}: "))
    if pasos >= 8000:
        dias_activos += 1  # cuenta solo cuando ocurre
el evento
print(f"Días activos: {dias_activos} de 7")
```

```
Pasos del día 1: 9500
Pasos del día 2: 4300
Pasos del día 3: 8000
Pasos del día 4: 12000
Pasos del día 5: 7999
Pasos del día 6: 3000
Pasos del día 7: 10100
Días activos: 4 de 7
```

Acumulador de suma: la cuenta del mercado

mercado.py

```
total = 0  # acumulador de suma: empieza en 0
for i in range(1, 4):
    precio = float(input(f"Precio del producto {i}:
"))
    total += precio
print(f"Total sin IVA: {total:.2f}")
print(f"Total con IVA: {total * 1.19:.2f}")
```

```
Precio del producto 1: 4500
Precio del producto 2: 12900
Precio del producto 3: 3200
Total sin IVA: 20600.00
Total con IVA: 24514.00
```

Vuelta	precio	total
antes	–	0
1	4500.0	4500.0
2	12900.0	17400.0
3	3200.0	20600.0

La tabla de la izquierda es una **prueba de escritorio**: el valor de cada variable al final de cada vuelta.

Acumulador de producto: potencia sin usar **

potencia.py

```
base = int(input("Base: "))
exponente = int(input("Exponente: "))
resultado = 1  # acumulador de producto: empieza en 1
for _ in range(exponente):
    resultado *= base
print(f"{base}^{exponente} = {resultado}")
```

```
Base: 3
Exponente: 4
3^4 = 81
```

¿Y si resultado empezara en 0? Todo daría 0: 0 multiplicado por cualquier número es 0.

4 · Recorrer textos y listas

El `for` también recorre **cualquier secuencia**: con un texto, la variable toma **un carácter por vuelta**; con una lista, **un elemento por vuelta**.

letras.py

```
for letra in "Lógica":
    print(letra, end=" ")
```

```
L ó g i c a
```

materias.py

```
materias = ["Lógica", "Cálculo",
"Física"]
for m in materias:
    print("Estudio", m)
```

```
Estudio Lógica
Estudio Cálculo
Estudio Física
```

`print(x, end=" ")` escribe un espacio en lugar de saltar de línea: varias vueltas quedan en la **misma línea**. Un `print()` vacío hace el salto.

clave.py

```
clave = input("Crea una contraseña: ")
mayusculas = 0
digitos = 0
for c in clave:  # un carácter por vuelta
    if c.isupper():
        mayusculas += 1
    elif c.isdigit():
        digitos += 1
if len(clave) >= 8 and mayusculas >= 1 and digitos >=
1:
    print("Contraseña segura")
else:
    print("Débil: usa 8 o más caracteres, una
mayúscula y un número")
```

```
Crea una contraseña: Medellin2026
Contraseña segura
```

Herramienta	Ejemplo
`len(x)`	`len("hola")` → `4`
`c.isupper()`	`"A".isupper()` → `True`
`c.isdigit()`	`"7".isdigit()` → `True`
`c.isalpha()`	`"ñ".isalpha()` → `True`
`texto.lower()`	`"HoLa".lower()` → `"hola"`
`c in "aeiou"`	`"e" in "aeiou"` → `True`

5 · Ciclos anidados

Un ciclo **dentro** de otro: por cada vuelta del ciclo externo, el interno se ejecuta **completo**, desde el principio (como un reloj: por cada hora pasan los 60 minutos). Si el externo da A vueltas y el interno B, el cuerpo interno se ejecuta **A × B** veces.

sala.py — puestos de una sala de cine

```
for fila in "ABC":  # externo: 3 vueltas
    for asiento in range(1, 6):  # interno: 5 vueltas
por fila
        print(f"{fila}{asiento}", end=" ")
    print()  # salto de línea al terminar la fila
```

```
A1 A2 A3 A4 A5
B1 B2 B3 B4 B5
C1 C2 C3 C4 C5
```

3 filas × 5 asientos = 15 puestos. Si el `print()` vacío tuviera 8 espacios de sangría, el salto ocurriría después de cada asiento.

triangulo.py — el interno depende del externo

```
n = int(input("Altura: "))
for i in range(1, n + 1):
    for j in range(i):  # la fila i tiene i asteriscos
        print("*", end="")
    print()
```

```
Altura: 4
*
**
***
****
```

El interno da 1, 2, 3 y 4 vueltas: 10 asteriscos en total.

6 · Errores frecuentes

Error	Ejemplo	Causa y corrección
SyntaxError	`for i in range(5)`	faltan los dos puntos.
IndentationError	`for c in "hola":` `print(c)`	el cuerpo necesita 4 espacios de sangría.
TypeError	`n = float(input())` `for i in range(n):`	`range()` solo acepta enteros: lee con `int()`.
Error lógico	`total = 0` dentro del ciclo	se reinicia en cada vuelta: créalo antes del ciclo.
Error lógico	`range(1, 10)` para "del 1 al 10"	la parada no se incluye: `range(1, 11)`.
Error lógico	`producto = 0`	todo da 0: el acumulador de producto empieza en 1.
Error lógico	`print(suma)` con sangría	se imprime en cada vuelta: sácalo del ciclo.

7 · Practica sin computador

¿Cuántas vueltas da?

Ciclo	Valores que genera	Vueltas
`for i in range(2, 9):`		
`for i in range(1, 20, 4):`		
`for i in range(10, 0, -3):`		
`for i in range(-3, 3):`		
`for letra in "Python":`		

Pruebas de escritorio: escribe el valor de cada variable al final de cada vuelta

A

```
total = 0
for i in range(1, 6, 2):
    total += i * 2
```

Vuelta	i	total
1		
2		
3		

B

```
r = 1
for k in range(4, 0,
-1):
    r *= k
```

Vuelta	k	r
1		
2		
3		
4		

C

```
cuenta = 0
for n in range(10, 30,
4):
    if n % 3 == 0:
        cuenta += 1
```

Vuelta	n	cuenta
1		
2		
3		
4		
5		

Resumen para tomarle foto 📸

Repetir N veces
`for _ in range(n):`

De a hasta b
`range(a, b + 1)`

Hacia atrás
`range(b, a - 1, -1)`

Contador
`c = 0` … `c += 1`

Acumulador
`t = 0` … `t += x` · `p = 1` … `p *= x`

Misma línea
`print(x, end=" ")` · `print()`

8 · Caja de herramientas para los ejercicios

Quiero...	En Python	Ejemplo
aumentar un saldo un %	`s += s * tasa / 100`	`1000000 al 2 % → 1020000.0`
el 19 % de un valor	`valor * 0.19`	`80000 * 0.19 → 15200.0`
sumarle el 19 %	`valor * 1.19`	`80000 * 1.19 → 95200.0`
qué % es una parte	`parte / total * 100`	`3 / 5 * 100 → 60.0`
¿es divisible?	`n % d == 0`	`12 % 4 → 0`
mostrar 2 decimales	`f"{x:.2f}"`	`f"{2/3:.2f}" → 0.67`
reservar ancho	`f"{x:4d}"`	`f"{7:4d}" → "  7"`
repetir un texto	`" " * 3`	`"ab" * 2 → "abab"`

Antes de programar cada ejercicio responde: ¿cuántas vueltas da el ciclo y qué valores toma su variable? ¿Qué contadores o acumuladores necesito y dónde se crean? ¿Qué se hace dentro del ciclo y qué después? Luego compruébalo en el 🐍 **Laboratorio** de la lección interactiva (se corrige solo) o en IDLE / Thonny.

9 · Ejercicios propuestos

12 ejercicios de menor a mayor dificultad. Bosqueja tu programa en el espacio y pruébalo con los ejemplos y con casos límite (0, 1, el valor exacto de un tope).

🟢 Nivel 1 · Calentamiento — Un ciclo con un contador o un acumulador y un cálculo con porcentaje.

Nivel 1 · for + range

1. Lista de precios con IVA

Una papelería quiere pegar en la vitrina una lista con el precio de comprar **1, 2, 3... hasta N unidades** de un mismo producto, **ya con el IVA del 19 %**. Pide el precio unitario sin IVA (real) y la cantidad máxima N (entero), y muestra una línea por cada cantidad con el formato `k unidades: X` (2 decimales).
Entrada: un real y un entero · **Salida:** N líneas
Ejemplo: 10000 y 3 → `1 unidades: 11900.00` · `2 unidades: 23800.00` · `3 unidades: 35700.00`.
💡 **Pista:** piensa qué valores debe tomar la variable del ciclo (¿desde cuál y hasta cuál?) y recuerda que `range` nunca incluye su valor de parada.

Nivel 1 · for + acumulador

2. Alcancía semanal

Durante varias semanas guardas dinero en una alcancía. Pide cuántas semanas N vas a registrar y luego, **semana por semana**, cuánto ahorraste (real). Al final muestra `Total ahorrado: X` y `Promedio semanal: X` con 2 decimales. Si N es 0 o negativo, muestra `No hay semanas para calcular`.

Entrada: un entero N y luego N reales · **Salida:** total y promedio

Ejemplo: 4 semanas: 20000, 15000, 0, 25000 → `Total ahorrado: 60000.00` · `Promedio semanal: 15000.00`.

💡 **Pista:** necesitas una variable que vaya juntando los valores; créala **antes** del ciclo con el valor que no altera una suma. ¿Qué pasaría con el promedio si N fuera 0?

Nivel 1 · for + contadores

3. ¿Cuántos aprobaron?

El docente quiere un resumen rápido de un parcial. Pide la cantidad de estudiantes N (entero mayor que 0) y luego la nota de cada uno (real de 0 a 5). Una nota aprueba si es de **3.0 o más**. Muestra `Aprobados: A`, `Reprobados: R` y `Porcentaje de aprobación: P%` con 2 decimales.

Entrada: un entero N y N reales · **Salida:** tres líneas

Ejemplo: 5 notas: 3.0, 2.9, 4.5, 1.0, 3.5 → `Aprobados: 3` · `Reprobados: 2` · `Porcentaje de aprobación: 60.00%`.

💡 **Pista:** un contador cuenta **eventos**: decide dentro del ciclo qué evento ocurrió con cada nota. El porcentaje compara la parte con el total de estudiantes.

🟡 Nivel 2 · Intermedio — Acumuladores que se actualizan con su propio valor, recorrer textos y varios acumuladores.

Nivel 2 · for + acumulador con %

4. Interés compuesto mes a mes

Un CDT paga interés **compuesto** mensual: cada mes el saldo crece un porcentaje **sobre el saldo que había**. Pide el capital inicial (real), la tasa mensual en % (real) y el número de meses (entero). Muestra una línea por mes con `Mes k: saldo` y al final `Intereses ganados: X`, todo con 2 decimales.
Entrada: un real, un real y un entero · **Salida:** una línea por mes y los intereses
Ejemplo: 1000000, 2 y 3 meses → `Mes 1: 1020000.00` · `Mes 2: 1040400.00` · `Mes 3: 1061208.00` · `Intereses ganados: 61208.00`.
💡 **Pista:** el saldo es un acumulador que en cada vuelta se actualiza a partir de su propio valor anterior. Los intereses ganados se pueden obtener al final, sin un segundo acumulador.

Nivel 2 · for sobre un texto

5. Analizador de frases

Pide una frase y recórrela **letra por letra** para contar cuántas **vocales** (a, e, i, o, u, también con tilde o diéresis y en mayúscula) y cuántas **consonantes** (cualquier otra letra, incluida la ñ) tiene. Los espacios, números y signos no cuentan. Muestra `Vocales: V`, `Consonantes: C` y `Porcentaje de vocales: P%` (sobre el total de letras, 2 decimales). Si no hay letras, muestra `La frase no tiene letras`.
Entrada: un texto · **Salida:** tres líneas o el aviso
Ejemplo: `Hola Mundo` → `Vocales: 4` · `Consonantes: 5` · `Porcentaje de vocales: 44.44%`.
💡 **Pista:** un `for` sobre un texto te entrega un carácter por vuelta. Para no preocuparte por mayúsculas, puedes convertir la frase a minúsculas antes; para saber si un carácter es letra existe un método de los textos que responde True o False.

Nivel 2 · for + varios acumuladores

6. Factura del supermercado

Pide cuántos productos distintos lleva el cliente (N) y, para cada uno, su **precio unitario** (real) y la **cantidad** (entero). Si el subtotal de la compra **supera 200000**, se aplica un **descuento del 10 %**. El **IVA del 19 %** se calcula sobre el valor ya descontado. Muestra `Unidades: U` (total de unidades), `Subtotal: X`, `Descuento: X`, `IVA: X` y `Total: X`, con 2 decimales los valores en dinero.
Entrada: un entero N y N parejas (real, entero) · **Salida:** cinco líneas
Ejemplo: 2 productos: 50000 × 3 y 12000 × 5 → `Unidades: 8` · `Subtotal: 210000.00` · `Descuento: 21000.00` · `IVA: 35910.00` · `Total: 224910.00`.
💡 **Pista:** dentro del ciclo solo se acumula (dinero y unidades); el descuento y el IVA dependen del subtotal **completo**, así que van después del ciclo.

🟠 Nivel 3 · Avanzado — Tablas de pagos, divisores y los primeros ciclos anidados.

Nivel 3 · for + acumuladores + %

7. Plan de pagos de un crédito

Un crédito se paga con **abono fijo a capital**: cada mes se abona la misma parte del monto (monto ÷ meses) y, además, se paga el **interés del saldo pendiente** de ese mes. Pide el monto (real), la tasa mensual en % (real) y los meses (entero). Para cada mes muestra `Mes k | Interés: X | Cuota: X | Saldo: X` (cuota = abono + interés; saldo = lo que queda después del abono) y al final `Total intereses: X`, todo con 2 decimales.
Entrada: un real, un real y un entero · **Salida:** una fila por mes y el total
Ejemplo: 1000000, 1.5 y 4 meses → `Mes 1 | Interés: 15000.00 | Cuota: 265000.00 | Saldo: 750000.00` … `Mes 4 | Interés: 3750.00 | Cuota: 253750.00 | Saldo: 0.00` · `Total intereses: 37500.00`.
💡 **Pista:** en cada vuelta el interés se calcula con el saldo **antes** de abonar; el orden de las instrucciones dentro del ciclo cambia el resultado.

Nivel 3 · for + condición + acumulador

8. Divisores y números perfectos

Los griegos clasificaban los números según la suma de sus **divisores propios** (los que lo dividen exactamente y son menores que él): si la suma es igual al número es **perfecto**, si es mayor es **abundante** y si es menor es **deficiente**. Pide un entero n mayor que 1 y muestra `Divisores: …` (en una sola línea, separados por espacios), `Suma: S` y la clasificación (`Perfecto`, `Abundante` o `Deficiente`).
Entrada: un entero n > 1 · **Salida:** tres líneas
Ejemplo: 12 → `Divisores: 1 2 3 4 6` · `Suma: 16` · `Abundante`.
💡 **Pista:** prueba como posible divisor cada número desde 1 hasta n − 1; el operador que da el residuo de una división te dice si la división es exacta. Para imprimir varios valores en la misma línea revisa el parámetro `end` de `print`.

Nivel 3 · ciclos anidados

9. Tabla de multiplicar N × N

Pide un entero n (de 1 a 12) y dibuja la tabla de multiplicar de n × n: la fila i contiene los productos de i por 1, 2, …, n, **en la misma línea**, cada uno alineado a la derecha en 4 espacios. Debajo, muestra `Suma de la diagonal: S`, donde la diagonal son los productos de cada número por sí mismo (1×1, 2×2, …).
Entrada: un entero n · **Salida:** n filas y la suma
Ejemplo: 3 → `1 2 3` · `2 4 6` · `3 6 9` · `Suma de la diagonal: 14`.
💡 **Pista:** un ciclo recorre las filas y otro, **dentro**, las columnas. Piensa dónde debe ir el salto de línea y en qué momento un producto pertenece a la diagonal. El formato `{x:4d}` reserva 4 espacios.

🔴 Nivel 4 · Reto ★ — Ciclos anidados con formato exacto y acumuladores por grupo.

Nivel 4 · ciclos anidados

★ 10. Pirámide numérica

Pide la altura n (entero de 1 a 9) y dibuja una pirámide de números: la fila i (de 1 a n) empieza con **n – i espacios**, luego cuenta **de 1 hasta i** y después **regresa de i – 1 hasta 1**, todo pegado, sin espacios entre los dígitos.
Entrada: un entero n · **Salida:** n líneas
Ejemplo: 4 → `   1` · `  121` · `12321` · `1234321`.
💡 **Pista:** cada fila tiene tres partes y cada parte es un ciclo distinto dentro del ciclo de las filas. El ciclo que regresa necesita un `range` con paso negativo.

Nivel 4 · anidados + acumuladores + %

★ 11. Comisiones del equipo de ventas

Pide cuántos vendedores hay (V) y cuántos días se van a registrar (D). Para cada vendedor pide su **nombre** y luego la **venta de cada día** (real). La comisión de cada vendedor es del **5 %** de su total, o del **8 %** si su total llega a **1000000 o más**. Por cada vendedor muestra `Nombre: total T, comisión C`; al final muestra `Total del equipo: X`, `Total comisiones: X` y `Mejor vendedor: Nombre` (si hay empate, el primero). Valores con 2 decimales.
Entrada: dos enteros y, por vendedor, un texto y D reales · **Salida:** una línea por vendedor y tres de resumen
Ejemplo: 2 vendedores, 2 días: Ana 300000 y 750000; Luis 200000 y 100000 → `Ana: total 1050000.00, comisión 84000.00` · `Luis: total 300000.00, comisión 15000.00` · `Total del equipo: 1350000.00` · `Total comisiones: 99000.00` · `Mejor vendedor: Ana`.
💡 **Pista:** hay acumuladores que se reinician para **cada vendedor** y otros que acumulan para **todo el equipo**: piensa en qué lugar (antes de cuál ciclo) se inicializa cada uno. Para el mejor vendedor guarda el mayor total visto hasta el momento y de quién era.

Nivel 4 · anidados + formato

★ 12. Calendario del mes

Pide cuántos días tiene el mes (28 a 31) y en qué día de la semana cae el 1 (1 = lunes … 7 = domingo). Muestra el encabezado `Lu Ma Mi Ju Vi Sa Do` y debajo el calendario: cada día ocupa **2 caracteres** alineado a la derecha, las columnas se separan con **un espacio**, los huecos antes del día 1 son 2 espacios en blanco y cada semana termina en domingo (o en el último día del mes). Al final muestra `Fines de semana: X días` (sábados y domingos del mes).

Entrada: dos enteros · **Salida:** encabezado, semanas y conteo

Ejemplo: 31 días empezando en miércoles (3) → primera semana `      1  2  3  4  5`, segunda `6  7  8  9 10 11 12` … · `Fines de semana: 8 días`.

💡 **Pista:** lleva la cuenta de en qué columna (0 a 6) vas: así sabes cuándo saltar de línea y si el día es sábado o domingo. Los huecos iniciales son un ciclo aparte. Cuidado con el espacio separador: va **entre** columnas.